

RASTRO

Transparencia y Datos Abiertos Colombia — Hackathon Croma

REQUERIMIENTOS NO FUNCIONALES

Requerimientos No Funcionales

Cómo debe comportarse RASTRO — rendimiento, resiliencia, seguridad, cumplimiento legal y mantenibilidad.

Versión 1.0 — Agosto de 2026

Proyecto: RASTRO — motor de transparencia en contratación pública colombiana

Rendimiento

ID	Requerimiento	Cómo se cumple hoy
RNF-01	Las consultas repetidas al mismo documento/entidad deben responder de forma casi instantánea.	Caché en memoria con TTL de 10 minutos (MemoryCache) delante de cada método del puerto de datos.
RNF-02	Las llamadas a fuentes externas no deben bloquear indefinidamente una consulta.	Timeouts explícitos por llamada (30 s en el cliente MCP; 10–18 s en los servicios que agregan varias fuentes).
RNF-03	Una entidad de muy alto volumen de contratación no debe degradar la experiencia del resto del sistema.	Muestras acotadas (SAMPLE_CAP) al abrir el detalle de procesos, en vez de recorrer el histórico completo.

Disponibilidad y resiliencia

ID	Requerimiento	Cómo se cumple hoy
RNF-04	Una falla o límite de tasa de la fuente de datos externa no debe tumbar el servicio.	Patrón puerto/adaptador: dataSource.js degrada automáticamente de CromaAdapter a MockAdapter por método, de forma transparente para el resto del código.
RNF-05	El sistema debe poder reportar si está operando en modo degradado.	GET /api/meta expone degraded: boolean, calculado sobre una ventana móvil de las últimas 12 llamadas.
RNF-06	Las suscripciones y el historial de notificaciones deben sobrevivir a un reinicio del proceso.	Persistencia en archivos JSON (data/companies.json, data/outbox_emails.json, data/seen_opportunities.json).

Seguridad

ID	Requerimiento	Cómo se cumple hoy
RNF-07	Las contraseñas nunca deben almacenarse en texto plano.	Hash con bcrypt (factor de costo 10) antes de persistir.
RNF-08	El token de sesión no debe ser accesible por JavaScript del navegador.	JWT guardado en cookie httpOnly, seteada server-side por los route handlers de Next.js; el navegador nunca la lee directamente.
RNF-09	El acceso a datos personales sensibles (estudio de seguridad) debe estar restringido a cuentas autenticadas.	Middleware requireAuth en todas las rutas de /api/personas/*; el expediente público de NIT no requiere autenticación por ser información pública.
RNF-10	Las llamadas del navegador a servicios externos deben pasar siempre por el backend, nunca directo.	El frontend solo llama rutas propias /api/*; las claves de Croma/Anthropic/email nunca se exponen como variables NEXT_PUBLIC_*.

Cumplimiento legal

ID	Requerimiento	Cómo se cumple hoy
RNF-11	El producto no debe presentarse como emisor de veredictos sobre personas o empresas.	Principio de diseño transversal: niveles de evidencia por fuente, nunca un score combinado; aplicado también al resumen generado por IA.
RNF-12	El manejo de datos personales sensibles debe advertir explícitamente sus implicaciones legales.	Aviso visible en el estudio de seguridad citando la Ley de Habeas Data (Ley 1581 de 2012) antes de cada consulta.
RNF-13	El registro de empresa debe verificar contra una fuente oficial, no solo aceptar datos autodeclarados.	El NIT se valida contra RUES (estado ACTIVA) antes de crear la cuenta.

Usabilidad y accesibilidad

ID	Requerimiento	Cómo se cumple hoy
RNF-14	La interfaz debe comunicar honestamente cuando un dato no está disponible, sin fingir un resultado.	Estados vacíos explícitos ("sin hallazgo", "sin datos suficientes", "no fue posible calcular") en vez de ocultar la sección.
RNF-15	El producto debe verse y funcionar correctamente en dispositivos móviles y de escritorio.	Frontend construido con Tailwind CSS v4 (mobile-first) y componentes responsivos en toda la interfaz.

Mantenibilidad y extensibilidad

ID	Requerimiento	Cómo se cumple hoy
RNF-16	Debe ser posible reemplazar la fuente de datos o el proveedor de correo/WhatsApp sin tocar la lógica de negocio.	Arquitectura de puertos y adaptadores: los servicios dependen de interfaces (GovDataSource, EmailPort, WhatsAppPort), no de implementaciones concretas.
RNF-17	El código debe documentar las decisiones no obvias en el lugar donde importan, no en documentos externos que se desactualizan.	Comentarios explicando el porqué (no el qué) junto a cada decisión de diseño no evidente en el propio archivo.

Observabilidad

ID	Requerimiento	Cómo se cumple hoy
RNF-18	Las degradaciones a datos de respaldo deben quedar registradas para diagnóstico.	Cada caída a MockAdapter se registra en consola con el método y los argumentos que fallaron.
RNF-19	El estado de la fuente de datos debe ser consultable en tiempo real.	Endpoint GET /api/meta expone dataSource (mock/croma) y degraded en cada respuesta.